

דו"ח מכין

ניסוי 2: עיבוד אותות בזמן אמת

יותם אידן וייס 207215179

יזהר פחימה 208066381

1. מעבדי DSP מסוגים floating point ו-fixed point.

1.1. מעבד DSP מסוג fixed-point הוא סוג של מעבד דיגיטלי העובד לרוב בארכיטקטורה של 16 ביט או 24 ביט. מעבד 16-ביט טיפוסי שומר את הנתונים, המקדמים והאותות כמספרים שלמים בדיוק של 16 ביט. כדי להתמודד עם חוסר הדיוק שעלול להיווצר בחישובים מורכבים, ערכי ביניים (כגון תוצאות של פעולות כפל) נשמרים בדיוק גבוה יותר של 32 ביט בתוך אוגרים פנימיים (Accumulators) של 40 ביט. שיטה זו נועדה להפחית שגיאות עיגול מצטברות במהלך החישובים.

מעבדי fixed-point מציעים יתרונות של עלות נמוכה, מהירות פעולה גבוהה ויעילות חומרתית ואנרגטית. הם צורכים פחות חשמל ודורשים פחות סיליקון בייצור, מה שהופך אותם לבחירה טובה וחסכונית עבור מוצרים בייצור המוני כמו מכשירי חשמל ביתיים, כוננים קשיחים, נגני שמע ומצלמות דיגיטליות. לעומת זאת, חסרונותיהם המרכזיים כוללים טווח דינמי מוגבל וסכנה מוגברת לשגיאות עיגול מצטברות עקב מגבלות הדיוק בחישובים, מה שמוביל למורכבות רבה יותר בפיתוח התוכנה, מאחר שהמתכנתים נדרשים לבצע התאמות קפדניות ולנהל את הנתונים באופן ידני כדי למנוע חריגות וגלישת נתונים.

מעבד DSP מסוג floating-point הוא מעבד אותות דיגיטלי המשתמש בשיטת חישוב שנועדה להרחיב באופן משמעותי את הטווח הדינמי של המספרים שאותם המערכת יכולה לייצג. מעבד מסוג זה עובד בארכיטקטורה של 32 ביט, ומחלק את ייצוג המספר לשני רכיבים: "מנטיסה" (חלק השבר) בת 24 ביטים, ו"אקספוננט" של 8 ביטים. המנטיסה מייצגת ערך שברי הנע בטווח שבין -1 ל- 1 , בעוד שהאקספוננט הוא מספר שלם הקובע כמה מקומות יש להזיז את הנקודה הבינארית ימינה או שמאלה כדי להגיע לערך האמיתי של המספר.

היתרון הגדול ביותר של מעבדים אלו הוא הטווח הדינמי שהם מספקים. טווח זה רחב, שמתכנני המערכת יכולים כמעט להתעלם ממגבלות של גודל הנתונים במהלך פיתוח המערכת. עובדה זו הופכת את המעבדים הללו לקלים מאוד לתכנות ולשימוש. בניגוד למערכות של נקודה קבועה בהן המתכנת חייב לבצע התאמות ידניות כדי למנוע גלישת נתונים או חריגות חישוביות, במעבדי נקודה צפה התהליך אוטומטי ופשוט יותר, מה שמוביל לרוב גם לביצועים כלליים גבוהים יותר.

חסרונות המרכזיים של מעבדי floating-point נובעים מהמורכבות החומרתית שלהם. ראשית, הם בדרך כלל יקרים יותר לייצור. שנית, הם מתאפיינים בצריכת חשמל גבוהה יותר, מה שעלול להוות חיסרון משמעותי במערכות הדורשות יעילות אנרגטית או כאלו המופעלות באמצעות סוללות.

1.2. המעבדים ממשפחת TMS320C55xx שייכים למשפחת מעבדי fixed-point.

2. פיתוח מסננים ספרתיים

2.1. ההשהייה שהרמוניה מסויימת ω_0 מקבלת במסנן בעלת תגובה להלם $H(j\omega)$ תלויה במסנן. אם מתקיים שפאזת המסנן תלויה לינארית בתדר, כלומר: $\angle H(j\omega) = k\omega$ אז הרמונית ω_0 מקבלת השהייה ששווה ל: $\angle \frac{H(j\omega_0)}{\omega_0}$. אם פאזת המסנן לא תלויה לינארית

בתדר, כלומר מתקיים $\angle H(j\omega) \neq k\omega$ אז הרמוניות שונות יקבלו השהיות שונות.

2.2. כמו שהסברתי מקודם פאזה לינארית הינה פאזה שמקיימת $\angle H(j\omega) = k\omega$ ופאזה לא לינארית היא פאזה שלא מקיימת את התנאי הזה. בפאזה לא לינארית כל הרמוניה עלולה לקבל השהייה שונה ומכך צורת האות תשתנה וזה מה שיגרום לעיוות.

2.3. על מנת שלמסנן דיגיטלי תהיה פאזה לינארית, הדרישה המרכזית שחלה על התגובה להלם שלו היא שהיא חייבת להיות סימטרית או אנטי-סימטרית סביב נקודת האמצע שלה.

2.4. למסנן IIR לא יכולה להיות פאזה לינארית. למסנן FIR לא יכולה להיות פאזה לינארית נובעת מהיעדר סימטריה בתגובה להלם שלו. בניגוד למסנן FIR שבו התגובה להלם מוגדרת באופן ישיר בתהליך התכנון ולכן קל לעצב אותה כך שתהיה סימטרית בין ימין לשמאל התכנון של מסנן רקורסיבי IIR מתבסס על קביעת מקדמי הרקורסיה ולא על התגובה להלם עצמה. מכיוון שהתגובה להלם של המסנן הרקורסיבי נוצרת כתוצאה מאותם מקדמים, היא מטבעה חסרת סימטריה בין צידה הימני והשמאלי, וחוסר סימטריה מוביל בהכרח לכך שהפאזה של המסנן תהיה תמיד לא לינארית.

2.5. משפחת המסננים IIR שנותנת פאזה הכי קרובה ללינארית הינה מסנני Bessel.

2.6. למשפחת המסננים מסוג Butterworth filters אין אדווה בתגובה לאמפליטודה.

2.7. למסנן Chebyshev מסוג 1 להופיע רק בpassband. למסנן Chebyshev מסוג 2 להופיע רק בstopband. במסנן אליפטי מורשית אדווה גם בpassband וגם בstopband.

2.8. הפרמטרים שצריך להגדיר למסנן Chebyshev הינם:

- סוג התגובה: האם יהיה מסנן Low-pass או מסנן High-pass.
- תדר הקטעון (cut-off frequency)
- אחוז האדווה בpassband
- מספר הקטבים של המסנן

2.9. משוואת המיפוי המעתיקה ממישור s למישור z הינה:

$$s = \frac{2}{T} \cdot \left(\frac{1 - z^{-1}}{1 + z^{-1}} \right)$$

2.10. אגדיר $s = j\omega_a, z = e^{j\omega_d}$ כלומר כיצד הציר המדומה במישור s (ציר התדר האנלוגי) ממופה אל מעגל היחידה במישור z (ציר התדר הספרתי).

ואציב:

$$j\omega_a = \frac{2}{T} \cdot \frac{1 - e^{-j\omega_d}}{1 + e^{j\omega_d}} = \frac{2}{T} \cdot \frac{e^{-\frac{j\omega_d}{2}} \cdot \left(e^{\frac{j\omega_d}{2}} - e^{-\frac{j\omega_d}{2}} \right)}{e^{-\frac{j\omega_d}{2}} \cdot \left(e^{\frac{j\omega_d}{2}} + e^{-\frac{j\omega_d}{2}} \right)} = \frac{2}{T} \cdot \frac{\left(e^{\frac{j\omega_d}{2}} - e^{-\frac{j\omega_d}{2}} \right)}{\left(e^{\frac{j\omega_d}{2}} + e^{-\frac{j\omega_d}{2}} \right)}$$

$$= \frac{2}{T} \cdot \frac{2j \sin\left(\frac{\omega_d}{2}\right)}{2 \cos\left(\frac{\omega_d}{2}\right)} = j \cdot \frac{2}{T} \tan\left(\frac{\omega_d}{2}\right)$$

ומכאן מתקבל:

$$\omega_a = \frac{2}{T} \tan\left(\frac{\omega_d}{2}\right)$$

2.11. הקשר בין תגובת ההלם בין המסנן האנלוגי השקול H_{equ} למסנן הספרתי בתדר ω_a הינו:

$$H_{equ}(\omega_a) = H_d(\omega_d = \frac{\omega_a}{F_s} = \omega_a \cdot T_s)$$

זה נכון בתנאי שהאותות האנלוגיים $x_a(t), y_a(t)$ מקיימים את תנאי נייקוויסט ביחד לתדר הדגימה F_s .

2.12. הקשר בין תגובת התדר של המסנן השקול H_{equ} לבין תגובת התדר של האב-טיפוס נקבע על ידי עיוות תדר. במשוואה הקשר הינו:

$$H_{equ}(f) = H_{proto}\left(\frac{F_s}{\pi} \tan\left(\frac{\pi f}{F_s}\right)\right) \quad . \quad 2.13$$

$$f_c^{equ} = \frac{F_s}{\pi} \tan\left(\frac{\pi f_c^{proto}}{F_s}\right)$$

$$\Omega_c = 2\pi f_c = 2\pi \cdot 400 = 800\pi \quad 2.14$$

$$H_{proto}(s) = \frac{1}{\left(\frac{1}{2\pi f_c^{proto}}\right)s + 1} = \frac{1}{\frac{s}{\Omega_c} + 1} = \frac{\Omega_c}{s + \Omega_c} = \frac{800\pi}{s + 800\pi}$$

$$s = \frac{2}{T} \cdot \left(\frac{1 - z^{-1}}{1 + z^{-1}}\right), F_s = 1000\text{Hz} \rightarrow s = 2000 \cdot \left(\frac{1 - z^{-1}}{1 + z^{-1}}\right)$$

$$\begin{aligned} H_d(z) &= \frac{800\pi}{2000 \cdot \left(\frac{1 - z^{-1}}{1 + z^{-1}}\right) + 800\pi} = \frac{800\pi \cdot (1 + z^{-1})}{2000 \cdot (1 - z^{-1}) + 800\pi(1 + z^{-1})} \\ &= \frac{800\pi \cdot (1 + z^{-1})}{(2000 + 800\pi) + z^{-1}(800\pi - 2000)} \end{aligned}$$

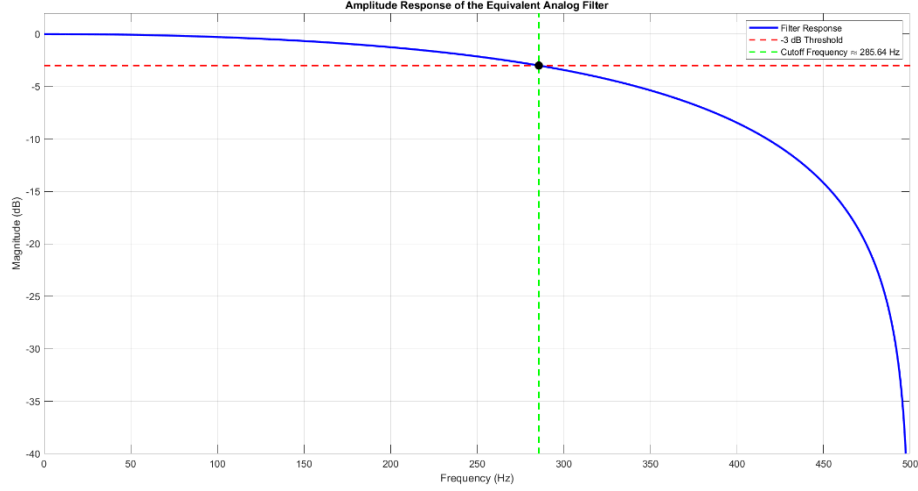
2.15. לא, תדר הקיטעון של המסנן השקול לא יהיה זהה לתדר הקיטעון של אב-הטיפוס. הסיבה לכך היא תופעת עיוות התדר המובנית בתוך ההתמרה הביילינארית. מכיוון שפיתחנו את המסנן ישירות מהאב-טיפוס ללא ביצוע *Pre-wrapping* תדר הקטעון נדחס כלפי מטה. מהפיתוח מקודם מתקבל:

$$f_{c, equ} = \frac{1}{\pi T} \arctan(\pi f_{c, proto} T)$$

אציב את הנתונים מהשאלה ואקבל:

$$f_{c, equ} = \frac{1}{\pi 0.001} \arctan(\pi 400 \cdot 0.001) = \frac{1000}{\pi} \arctan(0.4\pi) \approx 286 \text{ Hz}$$

2.16. התוצאה מהסימולציה ב *matlab* זהה לחישוב בפועל.



2.17. הראיתי מקודם שמתקיים:

$$\begin{aligned} f_{c, proto} &= \frac{1}{\pi T} \tan(\pi f_{c, equ} T) = \frac{F_s}{\pi} \tan\left(\frac{\pi f_{c, equ}}{F_s}\right) = \frac{1000}{\pi} \tan\left(\frac{400\pi}{1000}\right) \\ &= \frac{1000}{\pi} \tan(0.4\pi) \approx 980 \text{ Hz} \end{aligned}$$

2.18. אתחיל לפתח את המשוואות:

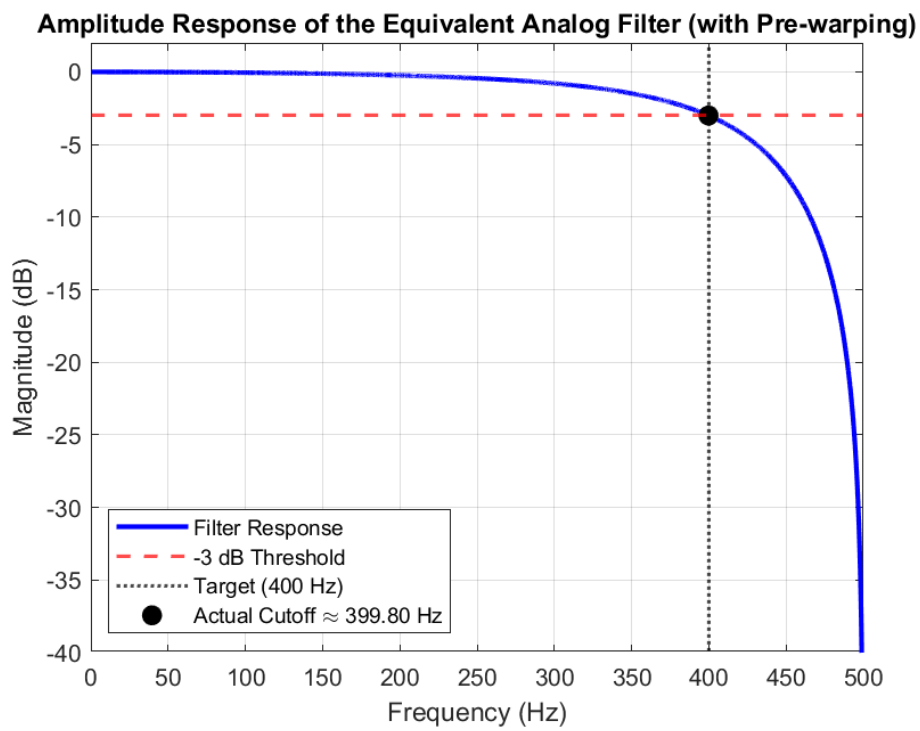
$$\Omega_{c, proto} = 2\pi f_{c, proto} = 2\pi \cdot 980 \approx 6158 \left[\frac{\text{rad}}{\text{second}} \right]$$

$$H_{proto}(s) = \frac{6158}{s + 6158}$$

$$s = \frac{2}{T} \cdot \left(\frac{1 - z^{-1}}{1 + z^{-1}} \right), F_s = 1000 \text{ Hz} \rightarrow s = 2000 \cdot \left(\frac{1 - z^{-1}}{1 + z^{-1}} \right)$$

$$\begin{aligned} H_a(z) &= \frac{6158}{2000 \cdot \left(\frac{1 - z^{-1}}{1 + z^{-1}} \right) + 6158} = \frac{6158 \cdot (1 + z^{-1})}{2000 \cdot (1 - z^{-1}) + 6158(1 + z^{-1})} \\ &= \frac{6158 \cdot (1 + z^{-1})}{8158 + z^{-1}4158} \end{aligned}$$

2.19



3. התכונות הייחודיות של מעבדי DSP בהשוואה למעבדים או בקרים אחרים

3.1. ללא יחידת DMA המעבד נאלץ להגיב לפסיקה עבור כל דגימה בודדת שמתקבלת או נשלחת מהתקני הקלט/פלט כדי לבצע את העברת הנתונים. תהליך זה מאלץ את המעבד לעצור את פעולתו הנוכחית, לשמור את מצבו הנוכחי, לעבור לטפל בפסיקה, לבצע אותה ולאחר מכן לשחזר את מצבו ולחזור לנקודת ההפסקה. בנוסף לפעולות אלו, נגרם חוסר יעילות מערכתי עקב ריקון צנרת הפקודות והחטאות בזיכרון המטמון מה שמייצר תקורת זמן משמעותית שבאה על חשבון ביצועי המעבד. יחידת DMA מתוכנתת לבצע את העברת הנתונים החוזרת ונשנית על *buffer* וממנו בצורה אוטומטית וללא כל התערבות של המעבד. במקום להפריע למעבד בכל דגימה, ה-DMA שולח פסיקה למעבד רק כאשר *buffer* שלם מתמלא או מתרוקן. בכך נחסכות מהמעבד המון פעולות ומשאביו מתפנים לטובת התמקדות נטו בחישובים האינטנסיביים של עיבוד האות על המידע הזמין, מה שמייצל ומגדיל את קצב העיבוד.

3.2. מסגרת (*frame*) הוא מושג המתאר קבוצה של דגימות עוקבות. עיבוד מבוסס מסגרות הינה שיטה של DSP שהמידע נאסף ומעובד במסגרות של מספר דגימות ולא כל דגימה בנפרד.

3.3. בעיבוד מבוסס דגימות המערכת פועלת באופן מיידי על כל דגימה בודדת. ברגע שה-ADC מפיק דגימה המעבד מקבל פסיקה, קורא את הדגימה, מבצע עליה את החישוב ושולח את התוצאה ל-DAC. היתרון הגדול בעיבוד זה הוא שההשהייה כמעט אפסית. לעומת זאת חיסרון מרכזי הינו חוסר יעילות ופגיעה בביצועי המעבד מכיוון שיש פסיקה עבור כל דגימה.

בעיבוד מבוסס מסגרות המערכת אוספת מסגרת בגודל N דגימות לתוך חוצץ בזכרון בעזרת DMA. רק כאשר המסגרת מתמלאת המעבד מקבל פסיקה בודדת ומעבד את כל מאות או אלפי הדגימות שבמסגרת במכה אחת. היתרונות הגדולים של שיטה זו היא יעילות חישובית גדולה מאוד והחיסרון המרכזי בשיטה זו היא שעלולה ליצור השהייה.

3.4. לפי הקוד נדרשים 8 חוצצים סך הכל כדי לעבד אות סטריאו. 2 ערוצי שמע (R, L) 2 ערוצי תעבורה- קליטה מהמיקרופון (Rcv) ושידור לרמקול ($Xmit$). 2 סטים של חוצצים- עבור כל ערוץ וכל כיוון. $8 = 2 \times 2 \times 2$ חוצצים סך הכל. לקליטה- $RcvL1, RcvR1, RcvL2, RcvR2$. לשידור- $XmitL1, XmitR1, XmitL2, XmitR2$.

שיטת "פינג פונג" היא טכניקה לעיבוד נתונים רציף. באודיו, הנתונים זורמים ללא הפסקה ואם יהיה עיכוב הוא ישר יישמע לאוזן. הרעיון בשיטת הפינג פונג הינו להפריד את העבודה בין בקר ה-DMA שאוסף את הדגימות לבין המעבד הראשי שמריץ אל האלגוריתמים כך שהם לעולם לא ימתינו זה לזה. מצב פינג- בקר ה-DMA מקבל דגימות חדשות מה-ADC וכותב אותם לתוך *buffer1*. באותו הזמן בדיוק המעבד לוקח את הדגימות המלאות והמוכנות שכבר נמצאות ב-*buffer2*, מעבד אותן ודוחף את התוצאה ל-*buffer* השידור של סט 2. מצב החלפה- ברגע שה-DMA מסיים למלא את סט 1 הוא שולח פסיקה למעבד שמודיעה למעבד שהוא סיים והגיע הזמן להתחלף. מצב פונג- ה-DMA מתחיל למלא את הנתונים חדשים את *buffer2* שהתפנה. במקביל המעבד ניגש ל-*buffer1* המלא והחדש ומתחיל לעבד אותו.

3.5. בעיות בעיבוד אותות הזקוקות לעיבוד מבוסס פריימים יכולות להיות זיהוי ועיבוד קול, דחיסת אודיו ווידאו, התמרות תדר וניתוח ספקטרלי, סינון רעשים אדפטיבי ועוד.

3.6. פעולת *macm* מבצעת במכה אחת:

- *Fetch* - ניגשת לזכרון ושולפת שני ערכים בו זמנית

- כפל- מבצעת כפל של שני המספרים
 - Accumulate- לוקח את תוצאת הכפל של שני המספרים ומחברת אותם לערך הקודם שהיה שמור בAC0.
 - Modify- האוגרים AR1, AR3 מקודמים כדי להצביע לתא הזיכרון הבא, החיצים זזים ימינה. זהו סימן ה+ בפעולה.
- 3.7. שני סוגי הפעולות במעבדי C55xx הם: *implied*- פקודות מובנות של המעבד. *Explicit*- פקודות מקבילות הנבנות ומוגדרות על ידי המשתמש.
- 3.8. יש זוג יחידות MAC במעבד. פקודת *mac* היא פקודה שניתנת לביצוע מקבילי.
- 3.9. הזזת דגימות בצורה שמתוארת חוברת לא יעילה במקרים בהם האות צריך להיות מהיר כי זאת בעולה מאוד בזבזנית שדורשת להעתיק נתונים מתא לתא בזיכרון.
- 3.10. החוצץ המעגלי הינה שיטה שבה שומרים את הנתונים במקומם ורק מחליפים את הדגימה הישנה ביותר (שבה אין כבר צורך) בדגימה החדשה ביותר. משתמשים במצביע שעוקב תמיד אחר המיקום של הדגימה החדשה ביותר וכאשר המצביע מגיע לקצה גבול הזיכרון שהוקצה לחוצץ, הוא לא ממשיך הלאה אלא קופץ חזרה להתחלה מה שיוצר של זיכרון במבנה מעגלי. במעבדי DSP המנגנון הזה מובנה בחומרה. זה חוסך זמן בטיפול בדגימות מכיוון שלא כל פעם מעתיקים את כל הדגימות ממקום למקום בזיכרון.
- 3.11. הסברתי ב3.10 למה החוצץ נקרא מעגלי.
- 3.12. הבדיקה $if (p==x)$ מתבצעת על מנת לבדוק האם המצביע p הגיע כעת לכתובת הגבול של המערך x .
- 3.13. חוצץ חומרתי יעיל מחוצץ תוכנתי בזמן של בדיקת הגבולות. בתוכנה התכנית חייבת לבדוק האם המצביע חרג מגבולות החוצץ וזה גוזל זמן. בחומרה המעבד מבצע את הקפיצה חזרה לתחילת המעגל באופן שמובנה במעבד. בנוסף זה יעיל מבחינת הזמן מכיוון שיכול לבצע את הפעולה במקביל.
- 3.14. יש 5 חוצצים מעגליים חמורתיים במעבד.
- 3.15. משמעות הסימן $||$ הוא ביצוע מקבילי. משמש לציון פקודות מקבילות מפורשת שבנו על ידי המשתמש.
- 3.16. אבצע חישוב של מחזורי שעון נדרשים על מנת את המסן: אתחול דורש 5 מחזורי שעון. פקודת *mov* מחזור שעון 1. בשורה שמופיעה $||$ סימן המקביליות פקודת הכפל *mpym* מתבצעת במקביל ל(*rpt SCR*) לכן 2 שורות אלה לוקחות מחזור שעון אחד. לולאת החישוב השנייה של פקודת ה-*macm* תרוץ בלולאה כדי לחשב את המקדמים ורצה $N-1$ פעמים. פקודת *macr* מחזור שעון אחד. *Mov* מחזור שעון אחד. סך הכל:
- $$5+1+(N-1)+1+1+1=N+8$$
- 3.17. בקוד 1 הלולאה רצה $N-1$ פעמים. התנאי והחיסור של $n-2$ מחזורי שעון. לאחר מכן הזזת הדגימות 8 מחזורי שעון. חישוב המסן (חישוב y) 9 מחזורי שעון. מחוץ ללולאה יש 5 מחזורי שעון. לכן סך הכל אקבל: $(2+8+9)*(N-1)+5=19N-14$
- 3.18. הטבלה המצורפת:

מספר מחזורי שעון לביצוע מסן FIR	שיטת תכנות
מסדר N	
$19N+5$	תוכנה בשפת C, הזזת הנתונים (אין <i>circular buffering</i>)

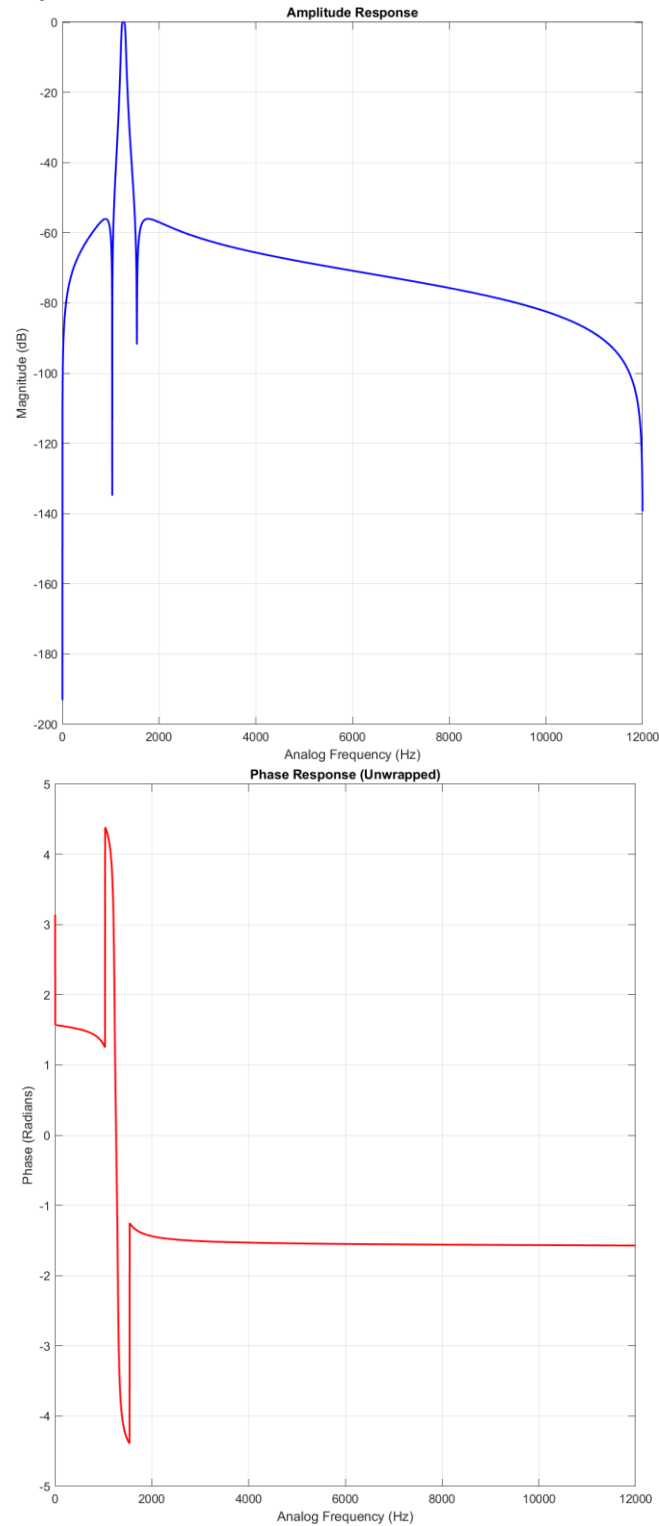
$N+8$	תוכנה באסמבלר, שימוש ב <i>circular buffer</i> המובנה בחומרת המעבד <i>DSP</i>
-------	---

3.19. המשאב של המעבד שעדיין לא נוצל הוא יחידת ה-*MAC* השנייה. הרעיון של השימוש במשאב זה הינו שבכל פעם אפשר לבצע חישוב של שתי דגימות מוצא במקביל. זה למעשה מכפיל את תפוקת החישוב של המעבד. במקום להשקיע N מחזורי זמן לכל דגימת מוצע משקיעים רק $N/2$ מחזורי שעון לכל דגימה.

4. השפיות האורך הסופי של המילה על עיבוד אותות ספרתי

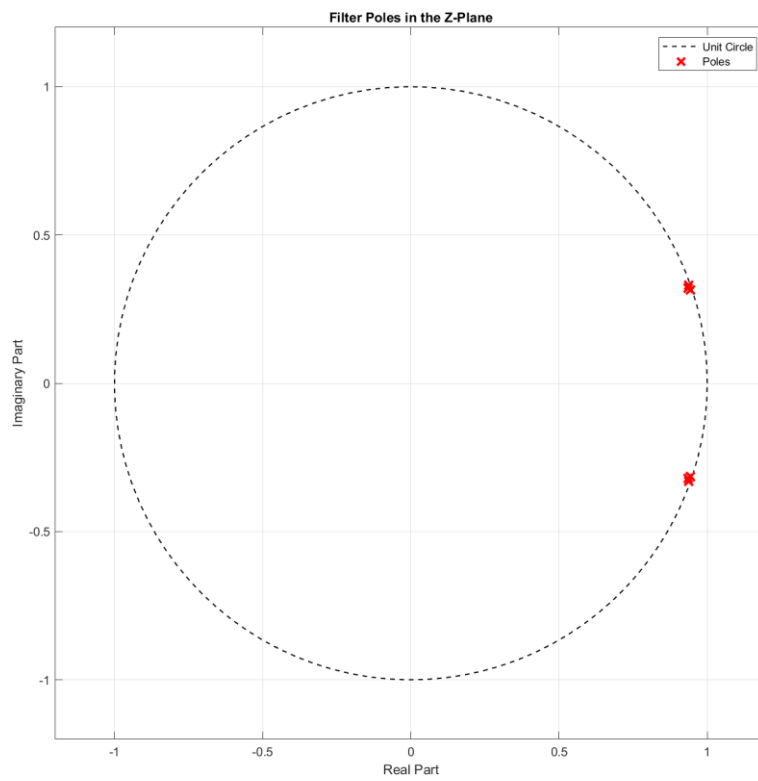
4.1. על פי קוד האסמבלי יש לכמת את המקדמים לאורך של 16 ביטים. יש צורך בכימות מכיוון שיש מגבלות חומרה. המעבד מתוכנן לעבוד בגדלים של *fixed point* במילים באורך 16 ביטים.

4.2. אצרך גרפים של תגובת האמפליטודה והפאזה של המסנן:

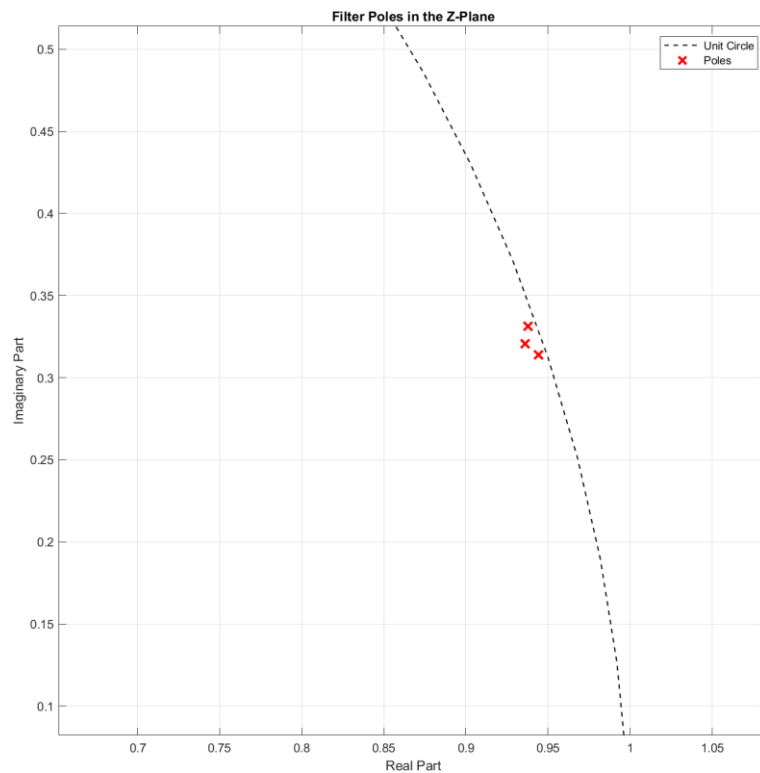


השתמשנו בפונקציית *unwrap* בשביל למנוע קפיצות מלאכותיות בפאזה.

4.3. המסנן הינו יציב אם כל הקטבים שלו נמצאים בתוך מעגל היחידה. אצורף תרשים של הקטבים:



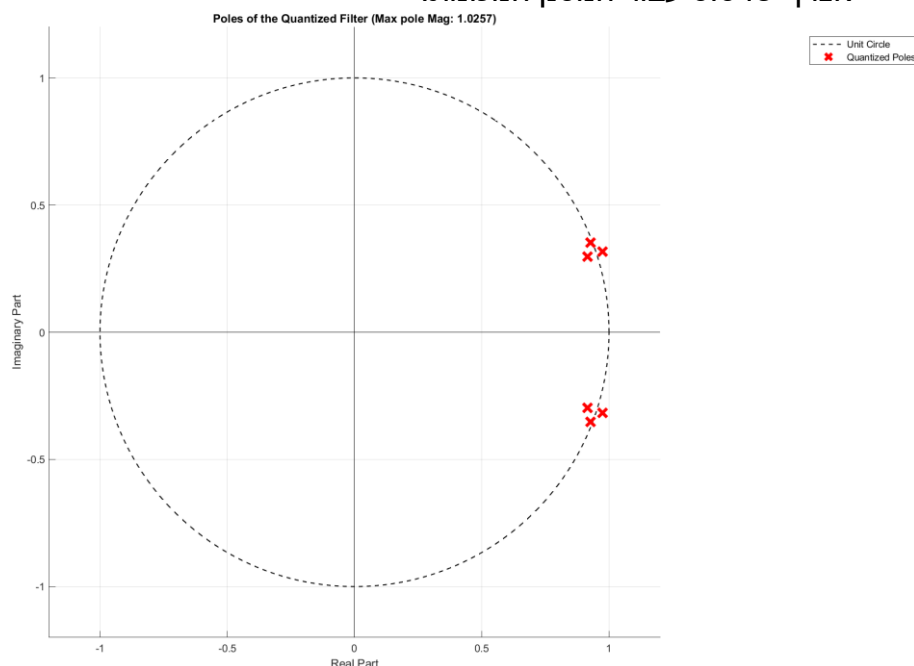
ובזום אין:



ניתן לראות שכל הקטבים נמצאים בתוך מעגל היחידה ולכן המסנן יציב.

4.4

אצורף שרטוט עבור המסנן המכומת:



ניתן לראות שיש קוטב שערך המגניטודה שלו גדול מ-1 לכן המסנן המכומת אינו יציב.
4.5 המשוואה ב-7.6.12 במקור 5 הינה:

$$\Delta p_i = - \sum_{k=1}^N \frac{p_i^{N-k}}{\prod_{\substack{l=1 \\ l \neq i}}^N (p_i - p_l)} \Delta a_k$$

מתארת מדד לרגישות של הקוטב ה- i לשינויים במקדמים $\{a_k\}$.

4.6 אסביר ואנמק באילו מקרים השיבוש בתפקוד המסנן מהותי יותר:

- השיבוש מהותי יותר כאשר סדר המסנן גבוה יותר. המכנה של המשוואה מורכב ממכפלה של ההפרשים בין הקוטב p_i לשאר הקטבים. ככל שסדר המסנן גבוה יותר כך יש יותר קטבים ויותר איברים במכפלה זאת. מכיוון שהקטבים של מסנן יציב נמצאים בתוך מעגל היחידה ערך המוחלט של המרחק לרוב קטן מ-1, והכפלה של מספר גדול של מספרים קטנים מ-1 תיצור מכנה מאוד קטן ולכן Δp_i יגדל.
- השיבוש כאשר הקטבים קרובים זה לזה יהיה גדול יותר מכיוון שהפרשים קטנים יותר יובילו לאותה תופעה בדיוק שהסברתי לעיל.
- שיבוש גדול יותר יהיה כאשר הקטבים קרובים יותר למעגל היחידה. במשוואה המונה p_i^{N-k} יהיה גדול יותר ככל שערכו המוחלט יהיה קרוב ל-1 וקטן יותר ככל שיהיה רחוק מ-1. מדבר על קטבים יציבים. על קטבים לא יציבים תזוזה קטנה המסנן יהפוך לפחות יציב ולכן ככל שמתרחקים ממעגל היחידה תהיה השפעה גדולה יותר.

4.7 באופן מעשי המימוש המועדף הוא מסנן בשלבים. במימוש בשלבים מפרקים את הפולינום המסנן הגדול למכפלה של פולינומים קטנים מסדר נמוך יותר מסדר שני. השורשים של המשוואה הריבועית תלויים רק בשלושה מקדמים סך הכל והרגישות שלהם לשגיאות כימות קטנה משמעותית. כתוצאה מזה תזוזת הקטבים הינה מינימלית ולכן המסנן הרבה יותר חסין להוצאת קוטב מאיזור היציבות.

4.8 הנזק הכי גדול שעלול להגרם למסנן עקב שגיאת כימות הינו אובדן יציבות.

5. כפל המספרים בייצוג *fixed-point*

- 5.1 *Fixed-point Q15* של מספרים הינה שיטה לייצוג של מספר שברי המורכב מ-16 ביטים. הוא נקרא Q15 מכיוון שיש בו 15 ביטים המייצגים את הגודל והביט הנותר משמש כביט סימן. טווח המספרים הינו $-2^{-15} \leq x \leq 1$. 15 ביטים הנדרשים לייצוג זה. טווח הערכים השלמים הינו מ-32,768 עד 32,768.
- 5.2 בשפת C טיפוס הנתונים התואם ל-Q15 הינו *int16* כי זה שומר ב-16 ביטים בזיכרון.
- 5.3 משוואת ההפרשים של המסנן הראשון הינה:

$$a_0 y_{current} + a_1 y_{previous} = b_0 x_{current} + b_1 x_{previous}$$

המספר ה"מקורי"	ייצוג Q15 של המספר	ייצוג בינארי 16 ביטים
$a_0 = 1.0$	32767	0111111111111111
$a_1 = -0.714159265358$	-23402	1010010010010110
$x_{current} = 0.129587958749$	4246	0001000010010110
$x_{previous} = 0.683493267002$	22397	0101011101111101
$b_0 = 0.5$	16384	0100000000000000
$b_1 = -0.157079632679$	-5147	1110101111100101
$y_{previous} = -0.286490745639$	-9388	1101101101010100
$y_{previous} = -0.247168891$	-8099	1110000001011101

- 5.4 בדיוק הערכים בטבלה למעלה בעמודה האמצעית.
- 5.5 הקוד המלא:

```
long temp = 0;
temp = temp + (long) b0*x_current;
temp = temp + (long) b1*x_previous;
temp = temp - (long) a1*y_previous;
temp = temp >> 15; /* divide by a0 */
```

משתנה *Long* הינו משתנה באורך 32 ביטים וכפל של שני משתנים בעלי 16 מחושב ב-32 ביטים בלי שגיאת קוונטיזציה. גודל משתנה הביניים בביטים צריך להיקבע לפי התוצאה שנשמרת בו, בין אם זה פעולת חיבור, כפל או שניהם.

- 5.6 השורה האחרונה בקוד הינה מכיוון שבפורמט Q15 התוצאה מכילה 30 ביטים של שבר כדי להחזיר את תוצאת הביניים לפורמט Q15 רגיל של 16 ביטים יש לבצע פעולת הזזה ימינה של 15 ביטים. זה שקול מתמטית לחילוק ב- 2^{15} שבדיוק הערך של a_0 .
- 5.7 אשלים את החסר בקוד:

```
y_current = (int) temp;
```

מכיוון שהמשתנה *y_current* הוגדר להיות מסוג *signed_int* כלומר באורך 16 ביטים מבצעים המרה חזרה מ-32 ביטים ל-16 ביטים.

5.8. בעיית *overflow* בחיבור מספרי *Fixed-Point* מתרחשת כאשר מחברים שני ערכים שסכומם חורג מהטווח המקסימלי שהמערכת מסוגלת לייצג. עקב השימוש בשיטת "משלים ל-2" לייצוג מספרים במעבד, חריגה זו גורמת למידע "לדרוס" את ביט הסימן. כתוצאה מכך סכום של שני מספרים חיוביים גדולים מתפרש על ידי המערכת כמספר שלילי.